

CSE 144 Final Project Report

Transfer Learning Challenge

Nigel Brigstocke, Cruz ID: 1978829

Kaggle public leaderboard: **0.96363 (2nd place)**

1 Introduction

Problem Goal and Setting:

1. This is a transfer learning challenge where we have 100 fine-grained images to classify with extremely limited training data, about 10 labeled images per class for training, totalling to 1079 training images. There are 1036 test images that the model will be evaluated on at the end. Predicted elements include planes, flowers, foods, and cars.
2. Because of the small amount and relative complexity of the training data, transfer learning is appropriate and necessary to perform well on the final task.
3. My final system uses the SigLIP-2 SoViT-400m backbone with 378 px to produce image embeddings, which I then use as input to a multivariable logistic regression model trained on those embeddings and the training data. I also used multi view test-time augmentation with center cropping and squashing. Then, I fuse that result with a zero-shot text signal from the SigLIP-2 text tower, calculating similarity to the class names I gave. I also had a Sinkhorn step where I assumed that the test set was balanced in the amount of data for each of the classes (roughly) and spread things apart that way. The results are very promising - it got about a 96% on 5 fold cross validation during training and it has also performed well on a portion of the kaggle test set - about 96.3%. I am currently in second place with only 10% of the test data being calculated with only one input attempt, to avoid overfitting myself to the small part of the test data we can see.

2 Dataset

1. There were 100 classes, 1,079 training images ($\approx 10-11$ per class) and 1,036 test images. Because of the already limited nature of the training data, I decided to do 5 fold cross validation instead of picking a specific part of the dataset to use.
2. I had a directory structure where my data folder had 100 folders for all of the different classes, and then an additional folder for the test data. The folders were named integers 0 to 99, corresponding with the number of classes - then had a function that could map the folder names to numbers to match class numbers.
3. I did a significant amount of preprocessing, though it is at test time - I did not store augmented images at any time. I resized images to SigLIP's 378 pixel input size. I then used a center crop, a full frame squash resize, both of which were also horizontally flipped.

3 Implementation

3.1 Model

1. The pretrain backbone I used was SigLIP-2 SoViT-400m, patch-14, 378px. This was simply because it could run on my 8GB of VRAM GPU and it was also the best out of many ones that I tested. I tested SigLIP (the first one), CLIP, and DINOv2, EVA-02, and ConvNeXt. The SigLIP was just the best at

the task, in all of the combinations that I tried. It has both an image tower for encoding and a tower for zero shot text prediction which I was able to take advantage of.

2. For architectural changes, kept the backbone entirely frozen and attached a single linear classifier, a heavily regularized multinomial logistic-regression probe, mapping the 1,152-dim embedding to 100 class logits. The probe is the only trained component, though I added on many components to the architecture. Firstly, I used both zero shot text prediction (mapped to class names) and the classifier on the image embeddings - I added these together to get an answer with different weights assigned to each, more weight towards the image embedding tower. The weights were a hyperparameter. Next, I also added a sinkhorn over the final test dataset to shift things slightly, assuming that there is a roughly equal amount of examples for each class in the test set. For this, I had to set a temperature for how strictly it should try to enforce the equal amount for each class rule.
3. All backbone layers are frozen so there is no fine tuning or unfreezing schedule. All layers are frozen. Fine tuning attempts at this model either failed to add any meaningful value or the overfit quickly.

3.2 Training

1. For my loss function and optimizer, the only thing I was training was a multinomial logistic regression model with L-BFGS from scikit-learn, which is convex with a single global minimum in terms of loss and nothing that could cause gradients to be weird.
2. This did mean that there were hyperparameters for me to set still. I have already mentioned the hyperparameters for weighting the text tower and image tower, as well as optimizing temperature for the sinkhorn due to the unknown nature of the test set. I also introduced regularization to the L-BFGS to avoid overfitting on the test set due to the large amount of parameters from the image embedding.
3. My environment was a single NVIDIA RTX 2080 (8GB) in a miniconda environment with PyTorch 2.5.1 + CUDA 12.. I used the timm library for getting backbones, and I used scikit-learn for the probe and Sinkhorn.

4 Experiments

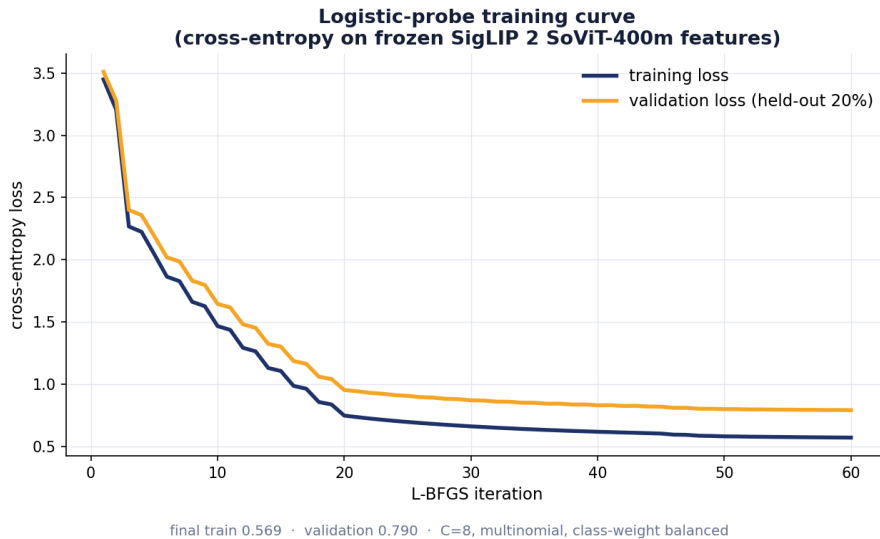
1. The first thing that I tried, setting up a baseline, was using ConvNeXt-Tiny, which reached about 76% cross fold validation, but I did not want to settle for that low score. I believe that it was overfitting on the small amount of training data.
2. For hyperparameter tuning and the validation method, I simply ran the model a multitude of times, looked at the results on 5 fold cross validation, and I checked to see how my different hyperparameter selections were affecting the final result. I started at 0.5 for the zero shot fusion rate but noticed that the text tower was oftentimes confidently wrong: thus, I lowered its weight and eventually settled on 0.35. A regularization strength of 8 from the L-BFGS model was chosen further based on cross validation, as was the temperature for the sinkhorn (when I made the held out portion have equal amounts).
3. I made many ablations. View them below:

Change vs. champion	Effect on CV accuracy
Squash-resize TTA view (added)	+1.6%
Zero-shot text fusion (added)	$\approx +2.6\%$
Transductive Sinkhorn balanced prior (added)	+1.5%
Bigger / higher-res backbones (SigLIP2-L@512, patch16@512)	≤ 0 (no gain)
Richer TTA (zoomed crop) / higher res 448–512	–0.2 to –0.6%
End-to-end fine-tuning of SigLIP-L	–1.5% (overfits)
Multi-backbone late fusion (add DINOv2/EVA-02/CLIP)	–3.8% (dilutes champion)

5 Results

The final pipeline reaches $\approx 96\%$ repeated-CV accuracy and 0.96363 on the Kaggle public leaderboard (2nd place).

1. Training / validation accuracy curves and plots:



We see that it fits slightly better to the training data, but the validation loss is pretty good.

2. Kaggle submission:

UCSC CSE 144 Spring 2026 Final Project

Submit Prediction ...

Overview Data Code Models Discussion **Leaderboard** Rules Team Submissions

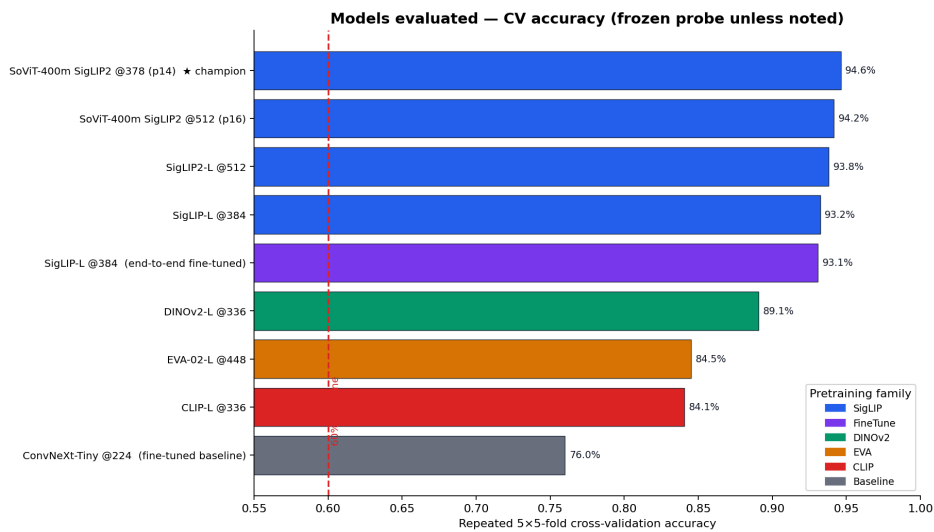
Public Private

This leaderboard is calculated with approximately 11% of the test data. The final results will be based on the other 89%, so the final standings may be different.

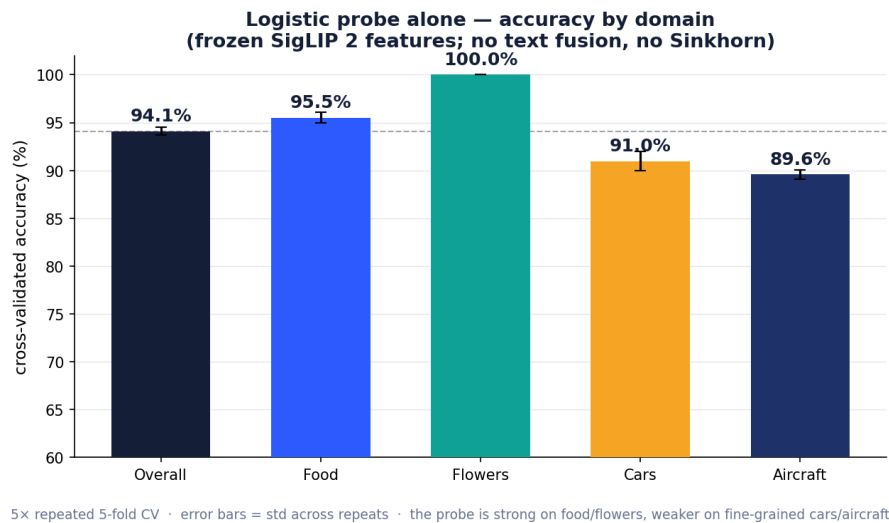
#	Team	Members	Score	Entries	Last	Join
1	Jay		0.98181	22	42m	
2	Nigel Brigstocke		0.96363	1	10d	

Your First Entry!
Welcome to the leaderboard!

3. Some error analysis:



This showed the error progression between different models



This showed that there were definitely classes where our model performed better and definitely classes where our model performed worse.

6 Discussion

1. There were a few moves that made all of the progress. First, choosing to do a completely frozen foundation model with a linear probe, specifically with SigLIP 2. Then, choosing to do the text tower that let me combine predictions from matching to the text vs matching to images via the logistic probe. Lastly, adding the sinkhorn to balance classes added a few percentage points on top of my already strong model.
2. In terms of failure cases, I noticed that overall fine tuning led to overfitting for this model. More models as an ensemble did not help. I also noticed that our model performs so much better on flowers and so much worse on aircraft, representing some bias in the model.

3. Some limitations are that the sinkhorn gain depends on the test set being balanced. Something else is that zero shot accuracy for text is currently matched with generated class names that were human verified, I need to ensure these class names are as accurate as possible. Future directions are making things more specific for sections of the training data where our model struggles.

7 Reproducibility

1. A fixed seed of 42 is set across everything. The convex probe also has a single minimum, so there is no risk of getting to a different minimum even with a different random seed. Everything else is pretty deterministic.
2. Environment. Python 3 with PyTorch 2.5.1 + CUDA 12.1, timm, scikit-learn, open_clip / transformers (≥ 4.49 for the SigLIP-2 Gemma tokenizer), pandas, pillow. Full pin list in requirements.txt; install with `pip install -r requirements.txt`.
3. Commands:
 1. Extract frozen champion features:

```
python src/extract_features.py --backbone vit_so400m_patch14_siglip_378.v2_webli --batch_size 8 --tta multi --out_suffix _tta
```
 2. Cache the zero-shot text probabilities:

```
python src/zeroshot_siglip.py --names_csv data/class_names.csv
```
 3. Generate the final submission:

```
python src/predict_fused.py --w 0.35 --tau 0.4 --out outputs/submission_fused.csv
```

8 Team Contributions

I, Nigel Brigstocke, was the only contributor for this project. I performed the dataset analysis, backbone selection, data loading, the logistic model, multi-view TTA, zero-shot text fusion, the sinkhorn step, cross validation experiments, and this report.

9 References

- Zhai, X., et al. “Sigmoid Loss for Language Image Pre-Training (SigLIP).” *ICCV*, 2023.
arxiv.org/abs/2303.15343
- Tschannen, M., et al. “SigLIP 2: Multilingual Vision-Language Encoders.” 2025. arxiv.org/abs/2502.14786
- Cuturi, M. “Sinkhorn Distances: Lightspeed Computation of Optimal Transport.” *NeurIPS*, 2013.
arxiv.org/abs/1306.0895
- Wightman, R. “PyTorch Image Models (timm).” GitHub. github.com/huggingface/pytorch-image-models
- Pedregosa, F., et al. “Scikit-learn: Machine Learning in Python.” *JMLR*, 2011.
jmlr.org/papers/v12/pedregosa11a.html